

Introdução ao Playwright

O **Playwright** é um framework moderno de automação de testes criado pela Microsoft para testar aplicações web.

Ele permite automatizar interações com navegadores como:

- Chromium (Chrome, Edge)
- Firefox
- WebKit (Safari)

Principais características

- Execução rápida (arquitetura moderna)
- Suporte nativo a múltiplos navegadores
- Esperas automáticas (auto-wait)
- Testes paralelos
- Suporte a múltiplas linguagens:
 - JavaScript / TypeScript
 - Python
 - Java
 - .NET

Comparação: Selenium vs Playwright

O **Selenium** é mais antigo e muito usado no mercado, enquanto o **Playwright** é mais moderno.

1. Arquitetura

- **Selenium**
 - Usa WebDriver (comunicação via protocolo HTTP)
 - Depende de drivers (ChromeDriver, GeckoDriver)
- **Playwright**
 - Comunicação direta com o navegador
 - Não precisa gerenciar drivers manualmente

Resultado: Playwright costuma ser mais rápido e estável

2.Esperas (Waits)

- **Selenium**
 - Implícita
 - Explícita
 - Fluent Wait
- **Playwright**
 - Esperas automáticas (auto-wait)
 - Menos código e menos erros

3.Paralelismo

- **Selenium**
 - Mais complexo (Grid)
- **Playwright**
 - Paralelismo nativo

4.Facilidade de uso

- **Selenium**
 - Mais verboso
 - Mais configuração
- **Playwright**
 - API moderna
 - Mais simples e direto

5.Quando usar cada um?

✓ Use **Selenium** quando:

- Projeto já existe com Selenium
- Equipe já domina
- Necessidade de integração com ferramentas legadas

✓ Use **Playwright** quando:

- Projeto novo
- Precisa de velocidade e estabilidade
- Quer menos dor com waits e flakiness

Configuração do ambiente – Playwright com Java

O **Playwright** tem suporte oficial para Java e funciona muito bem com Maven ou Gradle.

1. Pré-requisitos

Antes de tudo, você precisa ter:

- Java JDK 11 ou superior
- Maven ou Gradle
- IDE (IntelliJ ou Eclipse)

2. Criar projeto Maven

```
mvn archetype:generate -DgroupId=com.exemplo \
-DartifactId=playwright-java \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

3. Adicionar dependência do Playwright

No `pom.xml`, adicione:

```
<dependencies>
  <dependency>
    <groupId>com.microsoft.playwright</groupId>
    <artifactId>playwright</artifactId>
    <version>1.43.0</version>
  </dependency>
</dependencies>
```

Estrutura básica (conceito importante)

No Java, o fluxo padrão é:

Playwright → Browser → Page

- Playwright → engine
- Browser → navegador
- Page → aba (onde você testa)

Exemplo com interação

```
page.navigate("http://localhost:8080");

page.fill("#nome", "João");
page.click("#btnSalvar");

String mensagem = page.textContent("#mensagem");

System.out.println(mensagem);
```

Exemplo de teste:

```
import com.microsoft.playwright.*;
import org.junit.jupiter.api.*;

public class TestePlaywright {

    static Playwright playwright;
    static Browser browser;
    Page page;

    @BeforeAll
    static void setup() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch();
    }

    @AfterAll
```

```
static void tearDown() {
    browser.close();
    playwright.close();
}

@BeforeEach
void criarPagina() {
    page = browser.newPage();
}

@Test
void deveAbrirPagina() {
    page.navigate("https://example.com");
    Assertions.assertTrue(page.title().contains("Example"));
}
}
```

Primeiro teste em Java

Crie uma classe:

```
import com.microsoft.playwright.*;

public class ExemploPlaywright {
    public static void main(String[] args) {
        try (Playwright playwright = Playwright.create()) {

            Browser browser = playwright.chromium().launch(
                new BrowserType.LaunchOptions().setHeadless(false)
            );

            Page page = browser.newPage();
            page.navigate("https://example.com");

            System.out.println(page.title());
        }
    }
}
```

```
        browser.close();
    }
}
```

Explicação linha a linha

1. Criação do Playwright

```
try (Playwright playwright = Playwright.create()) {
```

O que está acontecendo?

- `Playwright.create()` inicializa o motor do **Playwright**
- É o ponto de entrada da automação

Por que usar **try**?

Isso é um **try-with-resources** do Java:

- Garante que o Playwright será fechado automaticamente
- Evita vazamento de recursos (memória/processos)

Equivalente conceitual no Selenium:

- Inicializar o `WebDriver`

2. Abrindo o navegador

```
Browser browser = playwright.chromium().launch(
    new BrowserType.LaunchOptions().setHeadless(false)
);
```

O que está acontecendo?

- `playwright.chromium()` → escolhe o navegador (Chrome/Edge)
- `.launch()` → inicia o navegador

Parâmetro importante:

```
.setHeadless(false)
```

- `false` → abre o navegador visível
- `true` → roda em background (headless)

Para aula, sempre use `false` (os alunos veem o teste acontecendo)

3. Criando uma aba (Page)

```
Page page = browser.newPage();
```

O que está acontecendo?

- Cria uma nova aba no navegador

Conceito chave:

Aqui começa a automação real

Fluxo mental:

```
Playwright → Browser → Page
```

No **Selenium**, isso seria equivalente a abrir uma nova janela/aba com o driver

4. Acessando uma URL

```
page.navigate("https://example.com");
```

O que está acontecendo?

- Abre a página no navegador

Ponto importante:

- O Playwright **já espera automaticamente o carregamento**
- Não precisa de `Thread.sleep` ou waits explícitos

Aqui já aparece uma grande vantagem sobre Selenium

5. Pegando o título da página

```
System.out.println(page.title());
```

O que está acontecendo?

- Captura o `<title>` da página HTML
- Imprime no console

Exemplo de saída:

```
Example Domain
```

Isso já pode virar um assert em teste real

6. Fechando o navegador

```
browser.close();
```

O que está acontecendo?

- Encerra o navegador

Observação importante:

Mesmo sem isso:

- O `try` já fecharia tudo

Mas é boa prática deixar explícito (didático para alunos)

Resumo conceitual

Você pode explicar assim:

Etapa	O que acontece
Playwright.create()	inicia engine

chromium().launch()	abre navegador
newPage()	cria aba
navigate()	acessa site
title()	lê informação
close()	encerra
